

# GitHub Actions - AWS OIDC Authentication Incident Report

Terraform AWS High Availability Platform | Troubleshooting and Resolution

**Incident:** GitHub Actions could not assume the AWS IAM role using OIDC.

Could not assume role with OIDC: The web identity token provided could not be validated.

## 1. Environment

| Component      | Configuration                            |
|----------------|------------------------------------------|
| Repository     | Terraform-aws-high-availability-platform |
| CI/CD          | GitHub Actions                           |
| Cloud          | AWS - ap-southeast-2 (Sydney)            |
| Terraform      | 1.16.3                                   |
| AWS CLI        | 2.33.22                                  |
| Authentication | GitHub Actions OIDC -> AWS STS           |
| IAM Role       | GitHubActions-Terraform-Role             |

## 2. What Happened

The Terraform CI workflow originally completed formatting, initialization and validation successfully. OIDC authentication was then added so GitHub Actions could access AWS without storing long-lived AWS access keys. After the AWS credentials step was introduced, the workflow failed at **Configure AWS Credentials** before the Terraform checks could continue.

The repeated AWS STS error indicated that the web identity token could not be validated. Because this can be caused by several layers - the GitHub token, IAM trust policy, OIDC provider, signing keys, TLS certificate chain, or the credentials action - the incident was isolated one layer at a time.

## 3. Troubleshooting Performed

**AWS CLI check:** Confirmed AWS CLI 2.33.22.

**IAM role trust policy:** Verified the GitHub federated OIDC provider, sts:AssumeRoleWithWebIdentity, audience condition, and repository/branch subject condition.

**OIDC discovery:** Verified GitHub issuer and JWKS endpoints were reachable.

**JWKS:** Confirmed GitHub published RSA/RS256 signing keys.

**Credentials action:** Updated aws-actions/configure-aws-credentials from v5 to v6.3.0. The error initially remained, so the version alone was not proven to be the cause.

**Actual token claims:** Added a temporary safe diagnostic that decoded only non-secret header/payload fields; the full token was never printed.

**Subject comparison:** Confirmed the actual GitHub sub exactly matched the IAM trust policy subject.

**Signing key:** Token used RS256 and kid 38826b17-6a30-5f9b-b169-8beb8202f723; the exact kid existed in GitHub JWKS.

**TLS chain:** Verified the GitHub OIDC endpoint certificate chain and TLS 1.3 connection; certificate verification returned OK.

**OIDC thumbprint:** Calculated the relevant SHA-1 fingerprint and confirmed it matched the thumbprint already stored in AWS: ab9d0263244dd0326eb67015705a667e79cfe998.

**Direct AWS STS test:** Bypassed the credentials action and called aws sts assume-role-with-web-identity directly from the GitHub runner. AWS successfully returned an assumed-role ARN.

## 4. Key Diagnostic Evidence

The GitHub token contained the expected identity information:

```
alg: RS256
kid: 38826b17-6a30-5f9b-b169-8beb8202f723
iss: https://token.actions.githubusercontent.com
aud: sts.amazonaws.com
sub: repo:abdullahalfarabi150-a11y@278372942/Terraform-aws-high-availability-platform@1392697685
:ref:refs/heads/main
ref: refs/heads/main
```

The final direct STS test returned an assumed-role identity for **GitHubActions-Terraform-Role/GitHubOIDCTest**. In the same successful workflow run, Configure AWS Credentials, Terraform Format, Terraform Init, and Terraform Validate all completed successfully.

## 5. Resolution

The final working configuration used GitHub OIDC with **id-token: write**, an AWS IAM role trust policy matching the repository actual immutable OIDC subject, and **aws-actions/configure-aws-credentials@v6.3.0**. No long-lived AWS access key or secret key was required.

No incorrect thumbprint replacement or unnecessary IAM permission expansion was required. The existing provider thumbprint was verified as correct, and the direct STS test proved AWS could validate the GitHub token and assume the role.

**Root-cause limitation:** The available evidence does not prove one permanent configuration defect caused every earlier InvalidIdentityToken response. After the configuration and token were systematically verified, the same authentication path succeeded. A transient OIDC/STS validation or propagation condition is plausible, but it should not be presented as a confirmed root cause without AWS-side service evidence.

## 6. Final Workflow State

```
GitHub push to main
|
GitHub Actions runner
|
GitHub OIDC token
|
AWS STS AssumeRoleWithWebIdentity
|
GitHubActions-Terraform-Role
|
Terraform Format -> Init -> Validate
```

## 7. Lessons Learned

OIDC failures should be diagnosed layer by layer rather than by repeatedly expanding IAM permissions. The most useful checks were comparing the token actual aud/sub with the trust policy, confirming the token kid exists in the issuer JWKS, verifying the certificate chain/thumbprint, and finally bypassing the helper action with a direct STS call. The direct STS test was decisive because it demonstrated the complete GitHub-token-to-AWS-role authentication path.

Temporary diagnostic steps should be removed from the permanent CI workflow after troubleshooting.

## 8. Incident Outcome

| Check                           | Result |
|---------------------------------|--------|
| GitHub OIDC token generation    | PASS   |
| Issuer / audience / subject     | PASS   |
| IAM trust subject match         | PASS   |
| GitHub signing key in JWKS      | PASS   |
| TLS certificate verification    | PASS   |
| AWS OIDC thumbprint match       | PASS   |
| Direct AWS STS role assumption  | PASS   |
| Configure AWS Credentials       | PASS   |
| Terraform fmt / init / validate | PASS   |

**Suggested GitHub evidence:** keep the final successful GitHub Actions screenshot showing the direct STS test, Configure AWS Credentials, and Terraform checks all passing.